

The `-k gevent` bit instructs *gunicorn* to use *gevent* and its greenlets, and the `-w 4` tells it to use four worker threads (you should generally use no more than $n+1$ threads, where n is the number of cores your processor has — which is somewhat like the number of threads you tell *make* to spawn with the `-j` option). The `-b` option supplies the host-port pair (this can be omitted) and, finally, you have the application itself—the colon notation basically says *from MyWSGIApp import app* and use that.

You should put a reverse-proxy in front of the application —something like the *nginx* config below will do just fine:

```
server {
    listen 80;
    server_name _;

    location / {
        proxy_pass      http://127.0.0.1:5000/;
        proxy_redirect   off;

        proxy_set_header    Host            $host;
        proxy_set_header    X-Real-IP       $remote_addr;
        proxy_set_header    X-Forwarded-For $proxy_add_x_
forwarded_for;
    }
}
```

Ideally, all your static files should be served by *nginx*.

WSGI applications on Heroku

Heroku is apparently the next big thing in cloud technology. Its new Cedar stack makes it very easy to write WSGI applications and run them on the Web. With Heroku, you start with a *virtualenv* and develop your app like you generally would. When you're ready to push the app into Heroku, in the root directory of the application, issue the following command:

```
(venv)$: pip freeze > requirements.txt
```

Heroku recognises a Python app by the *requirements.txt* file, and it specifies all the dependencies of your application. Optionally, you can write a *runtime.txt* in which you specify which Python interpreter to use. Your *runtime.txt* file should include the following line:

```
python-x.y.z
```

...where *x.y.z* is the Python version. Supported versions are 2.7.3 and 3.3.0 (2.7.3 is the default runtime), but you can specify any publicly available version from 2.4.4. You can even use *pypy-1.9* if you want to. I couldn't find a way to use Stackless Python or PyPy 2.0 beta without writing

my own buildpack.

Remember that because Heroku's Procfile expects a single command to start the server, and because there are no reverse-proxies between the app and the Web (save for Heroku's routing mesh), your best bet is to use *gunicorn*.

WSGI applications on Google App Engine

Google App Engine defaults to using *CGI* for Python applications. To use WSGI, you must use the Python 2.7 runtime. Specify the following in your *app.yaml* file:

```
runtime: python27
api_version: 1
threadsafe: true
```

It's important to specify the thread safety of your app. If you're not sure, say 'false' —it doesn't hurt.

Specify the handlers as shown below:

```
handlers:
- url: /.+
  script: MyWSGIApp.app
```

End notes

Because WSGI is so nifty, I generally stick to Web frameworks that expose their apps as WSGI callables. Flask is a great framework if you want to write WSGI applications. Pylons is a lot more involved. Django does have its own server but it can also act as a first-class WSGI app, so you are covered.

I'll let you in on a secret. If you choose to use Green Unicorn and your app can handle static files, you don't even need to use a reverse-proxy—Green Unicorn is good enough to face the Web.

Maybe in another article, I'll explore how to actually write a WSGI application with WebOb and, maybe, Flask. I won't touch on writing raw WSGI applications in this article. It all hinges on demand, so if you want them, do pester *OSFY* with mails. 

References

- [1] WSGI and the Pluggable Pipe Dream - <http://lucmr.pocoo.org/2011/7/27/the-pluggable-pipedream/>
- [2] Selector - A WSGI URL-based router - <https://github.com/lukearno/selector/>
- [3] A bunch of useful WSGI libraries and middleware - <http://www.wsgi.org/en/latest/libraries.html>

By: Boudhayan Gupta

Describing himself as a 'retard by choice', the author believes that madness is a cure-all for whatever is wrong or right with society. A social media enthusiast, he can be reached at @BaloneyGeek on Twitter.